Open Dream-RSI: An Open-Source Library for Recursive Self-Improvement Around a Frozen LLM, with Replay-Gated Learned Policies and a Curated Knowledge Base

Patryk Orwat
Independent Researcher
patrykorwat@gmail.com

October 7, 2026

Abstract

We present **Open Dream-RSI**, an open-source (MIT) Python library that implements the exploration layer of recursive self-improvement (RSI) around a *frozen* large language model: no weight access, no GPU, no training. The library reproduces the discovery loop of Dream-RSI [1] and extends it in five directions that the paper leaves implicit. First, an *always-on supervisor* that self-schedules improvement cycles under an explicit API-call budget guard. Second, *LLM-written exploration policies*: the loop asks the model to write candidate expansion policies as Python source, executes them in an isolated subprocess, scores them by *counterfactual replay rollout* on the recorded discovery tree, and promotes them only on evidence. Third, a *knowledge curator* that distils verifier failures into short, schema-validated, deduplicated lesson records; each fresh lesson is a *candidate* intervention that reaches proposal prompts only after a paired-behavioral-replay gate proves it turns failures into solves without turning solves into failures. Fourth, *thought-conditioned branching*: each attempt records the one-line plan the model stated before its code, the proposal prompt carries the task's tried-idea ledger, and expansion leaves a branch as soon as its recorded idea repeats itself—semantic selection replacing purely numeric score-greedy choice. Fifth, a *structured Sentinel seam*: the error-class sentinel that annotates tool results in deployment becomes part of the world model itself—observations enter discovery-tree nodes in a replay-safe form (cross-session counters stripped by construction) and the reason an episode stopped (*declined* vs. *option removed*) is recorded as data, so counterfactual replay can tell them apart. We make performance claims on exactly **one benchmark**: goose × *TravelPlanner* (public, scored by the benchmark's own evaluators), where a choice-removal gate lifts preregistered eval-split plan delivery from 23% to 94% and final pass rate from 7.6% to 21.5% (exact McNemar p=1.4×10^{-17}), with all 91 undelivered cold episodes shown to die at the harness call cap. Everything else in the project—`bench` (54% API-call saving at equal solve quality) and `bench-policy` (a decoy-trap suite where score-greedy exploration solves **0/120** and replay-gated policies reach 77%)—is stated as a deterministic *machinery self-check*, a regression contract for gates, sandboxes and promotion logic, not a model-quality claim; we state this distinction explicitly. The package has zero runtime dependencies, ∼9.8k lines of library code, ∼5.4k lines of tests (359 passing), a live dashboard, and a single CLI.

Code: https://github.com/patrykorwat/open-dream-rsi — MIT license, Python ≥ 3.9, zero runtime dependencies, installable from source with no build step.

1 Introduction

Recursive self-improvement systems that post-train model weights [9] require GPU clusters, training infrastructure, and weight access. The Dream-RSI line of work [1] shows a lighter recipe: keep the model frozen and instead improve the *process around it*—an offline "dreaming" loop that optimises how the online agent exploits accumulated discovery history. Since the paper's release (2026-09-14) several independent implementations have appeared; Open Dream-RSI is one of them, not the first (see §9).

This library targets the gap between the paper's method sketch and a usable tool: a researcher with an OpenAI-compatible endpoint should be able to `pip`-install the loop, point it at their tasks and model, and get a self-improving search that is (a) budget-safe, (b) safe to run code the model writes, and (c) capable of getting *better*, not merely cheaper. We make five contributions beyond the published method description:

1. **Always-on supervisor with a hard budget guard.** `AutoRSIRuntime` self-schedules cycles, counts every API call (including offline dreaming), and refuses to overspend—the property any unattended deployment needs and no reference implementation ships.
2. **LLM-written exploration policies with replay-gated promotion.** Per task category, the loop asks the frozen model for a policy program (`choose_action(frontier, step)`), statically validates it, executes it in a scrubbed subprocess, scores it against the recorded discovery tree by counterfactual rollout, and promotes it only if it beats the incumbent on evidence (§3). Promotion, not trust, is the safety model.
3. **Knowledge curator: lessons that make iterations smarter.** Recipes and policies capture *what* worked; nothing captured *why* attempts failed, so a trap re-pays its tuition every run. The curator distils verifier failures into short validated lesson records and *promotes them by paired behavioral replay before any prompt ever sees them* (§4)—the skills/memory pattern of practical agent frameworks, applied to the loop itself.
4. **Thought-conditioned branching.** Every attempt records the one-line plan the model stated before its code; proposal prompts carry the task's tried-idea ledger, and expansion abandons a branch as soon as its recorded idea repeats itself—semantic selection replacing purely numeric score-greedy choice (§5).
5. **The Sentinel as a world-model layer.** The error-class sentinel that annotates failing tool calls in deployment (§6) stops being only a prompt channel: structured observations enter discovery-tree nodes through a replay-safe contract, and each episode records *why* it terminated—the decision-maker declining to continue versus the runtime removing the option—so counterfactual replay can distinguish the two, and LLM-written policies can read error-class recurrence as primitive frontier features (§6.1).

Evaluation policy: this paper makes performance claims on exactly **one benchmark** — goose × TravelPlanner, public and scored by the benchmark's own evaluators (§8) — under a protocol frozen before the eval pass. The deterministic, key-free suites `bench` and `bench-policy` (§7) are *machinery self-checks*: a scripted solver makes the model arm a constant, so their numbers are regression contracts over the library's own gates, sandboxes, promotion and retrieval logic, and are never presented as model-quality evidence.

2 System architecture

The package is ∼9.8k lines of pure-Python with no runtime dependencies (standard library only, including the LLM client) and ∼5.4k lines of tests. Modules:

- `core.tree` — discovery tree: hypothesis, verifier score, action history, and (when a Sentinel engine is wired) structured error-class facts and the episode's termination reason per node (§6.1).
- `core.agent` — thin LLM-agent abstraction steered by the currently promoted policy.
- `core.simulator` / `core.dreamer` — offline dreaming: replay and optimisation over recorded simulations without touching the online budget.
- `core.policygen` — policy generation, validation, sandboxed execution, counterfactual rollout scoring, promotion gate (§3).
- `core.curator` — lesson distillation, validation, retrieval, pruning (§4).
- `sentinel` — host-agnostic error-class engine plus the `SentinelObservation` world contract (§6).
- `memory` — persistent cross-run store: verified-solution recipes, promoted policies, `lessons.json`.
- `loop` — `AutoRSIRuntime`: scheduling, budget guard, feedback wiring between dreaming and online expansion.
- `llm` — OpenAI-compatible client (`POST {base_url}/chat/completions`) via `urllib`; works against OpenAI, vLLM, Ollama, LM Studio, or any protocol-compatible server through `OPENAI_BASE_URL`.
- `tools` — sandboxed code verification (same isolation boundary as the policy sandbox).
- `dashboard` — dependency-free live web dashboard.

Budget guard. Every call—online proposals, offline dreaming, policy generation, curation—increments a single counter checked before dispatch. A cycle halts at its budget rather than overspending; the supervisor can then sleep, re-plan, and resume from persisted memory. This is what makes the loop deployable unattended: the cost per cycle is a declared quantity, not a hope.

Sandbox boundary. Candidate policy code and model-generated solutions are never executed in-process. Execution happens in `python -I` subprocesses with a scrubbed environment (only `PATH`), a hard timeout that kills the child's entire process group (so backgrounded grandchildren cannot outlive it), POSIX address-space, CPU-second and thread-count limits, and a temp-dir harness. Invalid code is additionally rejected by AST-level static validation (shape: `def choose_action(frontier, step)`; no imports, no dunder access, no `__builtins__` reference—name plus constructed-key indexing is itself an escape route) before any process spawns. API keys never cross the sandbox wall.

2.1 Execution schemas

A common question for deployment is whether the loop must be a continuously-running background process. It need not be: all improvement state lives in the persistent memory directory (`policies.json`, `policy_codes.json`, `recipes.json`, `lessons.json`, `trees/`, `events.jsonl`),

so every execution mode below is resumable and interchangeable — run $N{+}1$ always starts where run N ended, regardless of who invoked it.

- **Daemon (supervisor owns the clock).** `odr loop -tasks tasks.json -interval 300` runs `run_forever`: wake → one budget-guarded cycle (online attempts + offline dreaming + curation) → sleep → repeat, until interrupted or `-cycles` is exhausted. Cost per cycle is bounded by `-budget` (default 20 calls), so an unattended daemon cannot overspend the endpoint.
- **Scheduled (cron/systemd own the clock).** `odr loop -once` executes exactly one cycle and exits. This is the recommended production mode: the scheduler provides restart-on-failure, logging, and calendar semantics, and memory persistence makes back-to-back `-once` invocations equivalent to the daemon without a resident process.
- **On-demand (a harness owns the clock).** `odr mcp` exposes the loop as five MCP-over-stdio tools (`odr_status`, `odr_recipes`, `odr_lessons`, `odr_add_task`, `odr_run_once`) to any MCP-speaking agent harness (Hermes, Goose, OpenCode, Claude Code, Zed). The harness spawns the server per session and *stops it with the session*; between tool calls the server is passive — it holds no timer and polls nothing. An everyday coding agent can therefore queue tasks for the dreamer and pull back verified recipes without any always-on component: improvement happens exactly when a harness (or cron) calls `odr_run_once`.
- **Offline tools.** `odr bench` (two-arm API-efficiency benchmark), `odr gate-replay` (applies the lesson promotion rule to recorded baseline/warm arm pairs from any replay harness, off-model), `odr status` (what memory has learned), and `odr dashboard` (live web view of the tree) run on demand against the same memory directory.

The three clocks are thus decoupled by design: the *learning loop* never needs a resident process, only a scheduler of some kind; the `events.jsonl` audit trail records which mode drove each cycle.

3 LLM-written exploration policies

For each task category the loop may request one LLM call producing a complete policy program. The candidate then passes a pipeline in which every stage can reject it:

1. **Static validation.** AST parse; required entry point present; banned constructs (imports, `eval`, dunder access) reject outright.
2. **Sandboxed execution** against recorded frontiers (above).
3. **Counterfactual rollout scoring.** The candidate's decisions are replayed on the recorded discovery tree: at each simulated step the candidate picks a frontier node; the reward is the score the child actually achieved in the recorded ladder (the node's own score when the branch was never expanded, its last recorded child when the ladder is exhausted). The rollout is *prefix-only*: at each step the candidate sees only descendants its own counterfactual path has revealed, and the `outcome` field is the best score among *revealed* descendants—no future score leaks into a present choice. One batched subprocess scores an entire rollout ($\sim$15$\times$ faster than per-step spawning). The objective is

$$\text{score}(\pi) \;=\; \frac{1}{T}\sum_{t=1}^{T} r_t \;+\; \beta_2 \cdot \frac{|\{\text{distinct picks}\}|}{T} \;-\; \beta_3 \cdot (1-e), \tag{1}$$

with $\beta_2 = 0.2$, $\beta_3 = 0.1$, and the discovery-efficiency factor $e = |\{(\text{node}, r_t) \text{ first harvests}\}|/T$: a step that re-pays a value already recorded from the same node replays an exhausted ladder — a call producing no new discovery. The term operationalizes Dream-RSI v2's rounds-per-call efficiency over a recorded tree [1] (an earlier latency formulation — rewarding early payoff — mis-ranked structural explorers below lucky ladder-order harvests on decoy fixtures and was rejected in measurement). The objective is applied *identically* to the candidate and to an in-process greedy baseline. Mean-only scoring mis-ranks explorers below greedy on trap trees; the diversity term is what lets a genuinely structural explorer win the gate.

4. **Promotion gate.** A candidate replaces the incumbent only with evidence: $\geq$80% valid sandbox decisions (`MIN_VALIDITY`=0.8) over a horizon of $\geq$8 recorded steps (`MIN_HORIZON`), and a rollout score strictly beating the greedy baseline under the shared objective (a tie is not evidence — the same strictness the lesson gate applies). When an incumbent exists it is *re-scored on the current tree* rather than trusted at its stored score: the replay world grows between cycles, and an old threshold would admit candidates that are genuinely worse than the incumbent on today's evidence.

5. **Call economy.** A category that already solves with its promoted policy is not re-asked; regeneration requires 4 new replay steps of evidence (`POLICY_REGROW_STEPS`), and all generation calls count against the same budget guard. A third, opt-in mechanism stops *spending* on regeneration altogether: an anytime-valid betting martingale per category over the binary "did this cycle's regeneration promote" outcome (null rate calibrated from the first rounds, floored at 0.5; λ=0.5, α=0.05), after whose alarm the last promoted policy stays active and further policy-generation calls are skipped [2]—the recorded-tree mapping of stopping rules for self-evolving loops. Benchmarks pin it off.

The promoted program then *steers online expansion*: which frontier node to expand next is decided by validated model-written code rather than a fixed heuristic. This closes the loop the paper describes—the process around the model improves—while every untrusted artefact (code and text alike) passes a structural gate.

4 The knowledge curator

Recipes (verified solutions as warm starts) and policies capture *what* worked. Neither captures *why* attempts fail, so the same trap re-buys its tuition fee every run. The curator adds a third persistent artefact.

Lesson records. After failed verification, the loop invokes the model once (one call, budget-counted) to distil the verifier evidence into lessons carrying a short trigger phrase, a lesson text of $\leq$3000 characters, and $\leq$5 evidence snippets.

Lessons are *pure text*—never executed—so their gate is structural plus evidential rather than sandboxed: schema and caps (2–4 trigger words; 20–3000 characters; $\leq$8 lessons per category; $\leq$4 lessons / 15000 characters injected per prompt; $\leq$5 evidence snippets each), evidence required at admission, and deduplication against the existing store.

Behavioral promotion: lessons must earn activation. A structural gate validates *form*; it cannot see that a well-formed lesson is harmful. A live-model replay on a production task family (a greedy-decoding 8B model, 20 held-out tasks, identical cached-page sandbox for every arm) made this concrete: lessons that passed every existing gate—true traps, honest counters, correct retrieval—took the solve rate from 18/20 to 6/20, because an unbounded "keep fetching before answering" instruction suppresses the answer decision on a model with no internal

stopping counter; even *content-neutral* injected text cost 5/20. A follow-up three-arm ablation localized the hazard precisely: the *same* content rewritten as declarative facts (no imperative verbs, neutral frame) scored 18/20—matching the contemporaneous cold baseline (exact McNemar p=1.0)—while the imperative form of those same true facts lost 12 paired comparisons with 0 wins ($p < 0.001$); an exemplar worked-example (recipe warm-start) arm scored 16/20 and a facts+exemplar arm 13/20, so imitation of a solved example does not anchor termination either. The hazard is the imperative mood and open-ended checking instruction, not the knowledge; the shipped prompt frames lessons as declarative background and the curator contract requires declarative phrasing. Trust is therefore not a gate and the curator does not use one. A distilled lesson is admitted as a *candidate intervention* with `status=staging`, invisible to proposal prompts (invariant enforced in `select_lessons`), and activates only after a paired behavioral replay: the loop snapshots the *first-failure* state of the task (branch code, verifier feedback, recipe *before* any fix leaks in) and rolls it out with and without the lesson to depth 2—a lesson's payoff is often path-level, a redirect at step 1 that solves at step 2. Activation requires the harm-asymmetric sign test $gains - regressions \geq 1$ with *zero* solve→fail regressions *and* an explicit stop clause in the text (a clause-less lesson is rejected regardless of the sample—an unbounded exploration instruction must never activate on a greedy solver whatever a small sample says). Rejected candidates never reach a prompt; activated lessons carry runtime `wins/uses` counters and auto-retire after $\geq K$ uses with zero wins. Replay calls are budget-counted and the gate runs only on spare budget, so knowledge never taxes the working loop. A third, upstream precondition follows from the seven-arm extension of this replay: abstract workflows induced from success trajectories, evidence certificates in the answer schema, reactive-on-error notes, and end-of-prompt framing of the measured-safe declarative text were each injected on the same family—every arm scored ZERO paired gains against the contemporaneous cold baseline (19/20 at 3.0 calls/solve), the worst arm significantly worse (11/20, p=0.008; recency made the model *act on* facts that were inert mid-prompt), and the single hard-tail task burned the full call budget in *all* arms, i.e. its failure was environment-side, not prompt-side. The loop therefore computes a *headroom verdict* per category from its own recorded outcomes (`headroom_verdict`: recent solve-rate < 0.7 or calls-per-solve > 5; empty history counts as headroom) and skips curator and gate calls entirely where the baseline already solves cheaply—guidance can only pay its perturbation tax there, and effort belongs to families with measured headroom.

Retrieval feedback. At proposal time the top *active* lessons for the category are retrieved (trigger-match, ranked by `wins/uses` counters updated from subsequent verification outcomes); counters drive ranking and eviction, so lessons that demonstrably stop helping are pruned automatically. The knowledge base therefore cannot silently grow stale, and its worst failure mode—injecting an unhelpful paragraph—is bounded twice: by the caps and by the promotion gate.

What is claimed and what is not. The scripted benchmark below demonstrates that the *wiring* works: a gated lesson, once promoted, causally changes what the proposal prompt contains and the scripted world responds to it. On the live replay above, the gate's decision rule—shipped as `odr gate-replay`, which joins baseline/warm arm records from any replay harness on a task key and applies `lesson_gate_verdict` off-model—rejects every harmful arm (net −13, −6, −5 against the cold baseline, zero gains in all three), so the documented harm could not reach a prompt under the shipped algorithm. What remains open is the *false-rejection* rate on a live model: no live arm in that suite had net wins to accept, so the accepting half of the gate is validated only on the scripted knowledge arm (§7), and lesson *writing* quality by a live model stays an open question this library deliberately does not claim to have settled (§10).

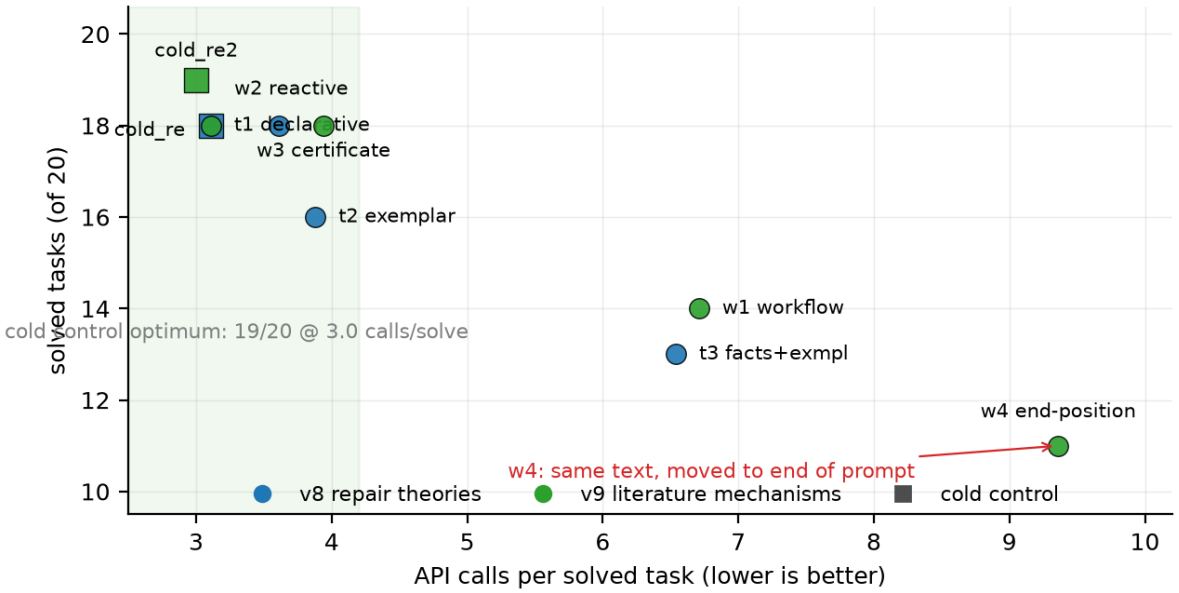


Figure 1: Injection-arm study on the live model (20 held-out tasks, greedy decode, one endpoint; v8 repair theories and v9 mechanisms each operationalize one published idea). Every arm is scored against the contemporaneous cold control run in the same sitting (squares). No arm beats cold; the gate rejects all of them. The end-position arm (w4) uses the *identical* text that is measured-safe mid-prompt (t1) — frame safety is position-dependent. Reactive-on-error injection (w2) pays zero prompt tax on clean episodes: the architecturally right channel, but with no benefit where cold is already optimal.

5 Thought-conditioned branching

Policies (section 3) select branches by recorded scalars only: the frontier observation carries scores, visit counts and error strings, but nothing about *which idea* a node stands for. A loop can therefore re-expand a branch forever without knowing it is re-polishing the same dead plan, and a proposal cannot see which ideas the branch already exhausted. Thought-conditioned branching (the runtime default, `enable_thoughts=True`, ablatable with `-no-thoughts`) closes this gap with three cheap, purely textual additions.

Per-node idea label. When enabled, the candidate system prompt asks the model for exactly one `PLAN:` line before the code block; the line is extracted tolerantly (bullets, dropped colons and case are forgiven) and stored as `TreeNode.thought`, truncated at 240 characters. Like lessons, thoughts are pure text—never executed, never fed to a sandbox—so their admission requires no gate beyond length, and they are persisted with the archived tree, so replay worlds and later cycles see the same labels. The frontier entry and the policy-generation summary gain a `thought` field, letting LLM-written policies reason over idea families, not just scores.

Tried-idea ledger. Every proposal prompt additionally carries the task's deduplicated idea ledger—best recorded outcome per distinct idea, $\leq$8 lines—rendered above the policy hints: which plans have already been attempted in this task, and what the verifier said about each. The model is instructed not to repeat a dead idea verbatim and to build on structurally promising ones. The ledger is deliberately task-wide rather than branch-local: whether an idea is dead does not depend on which node happens to be expanded next.

Semantic escape from idea stagnation. At expansion time, the loop applies the rule shown below after the promoted policy program declines to act and before the ε/greedy cascade decides:

Stagnation steering. Let b be the score-greedy frontier pick with idea t. If t is empty, or no recorded child of b carries t again (the branch is converging, not repeating), do nothing—normal cascade. Otherwise b is polishing a dead plan: choose the frontier node maximising $(\mathit{outcome}, -\mathit{children})$ among (i) nodes labelled with a *different*, non-empty idea, or, if none exist, (ii) nodes with no idea label at all (typically the seed, whose ladder may still hold untried ideas). No randomness, no API call, no verifier score improving: the trigger is recorded semantic evidence about the *idea*, and the selection prefers the least-expanded node with the best known continuation.

The rule strictly refines the existing machinery: when nothing is stagnant it is inert, and when it fires it can only redirect expansion to a node the policy cascade was already allowed to choose—so the worst outcome of a mislabelled thought is neutral, matching the fallback philosophy that protects the section-3 gate.

Causal attribution. The decoy-trap benchmark's thought arm (§7) isolates the contribution to the *expansion-time steering*, not to the prompt text: with ε forced to zero and both policy generation and curation disabled the arm escapes every trap from the first cycle (120/120, 164 calls, 1.37 calls per solve), while the `thought_steering=False` ablation (PLAN lines still recorded and the ledger still rendered into proposals, semantic pick disabled) collapses to the baselines—0/120 under greedy expansion, identical to ε-greedy's 29/120 with ε on. The scripted solver is deliberately no more generous to the thought arm than to the lesson arm: the ledger may redirect a repeatedly-polished dead idea to the alternative idea, never directly to the fix, and tests assert that every solved task's fix node grows from expanding the promising-idea node—the same ladder every other arm must climb. Selection therefore stops being purely numeric on evidence: the loop knows *which idea* a branch stands for, and abandons it when the record says the idea is dead.

6 The error-class Sentinel and its world-model seam

Curation above is bounded by the episode: it reads outcomes after episodes end and wakes on a schedule. Measured on a production agent's session store (30 days; anonymized replay fixture `fixtures/sentinel_audit.json`), a recurring error class re-appears *within* one session at a median gap of 4.3 minutes and *half* of all failing tool calls repeat a class the same session already suffered (Figure 2). No schedule wins that race, so the reactive half of the problem lives in the tool-execution layer.

The engine. `sentinel.py` is host-independent: `classify` recognises failures from status or payload; `normalize` strips URLs, paths, numbers and hex so that calls differing only by argument collapse to one *error class*; a SHA-1 signature keys the ledger. `SentinelEngine.observe()` counts recurrences per class per session and across sessions and emits at most one note per class per session — *declarative* text (recurrence facts plus an explicit stop condition, never a command; the replay arms of §4 showed imperative framing measurably extends loops). Clean calls pay zero prompt tax: the note rides only failing results. The same engine also carries a budget-pressure channel (*finalize nudge*: counts all calls, fires once per session at $\lceil 0.8 \cdot \mathit{budget} \rceil$, rides an ordinary tool result, never a blocking hook). Adapters ship for Hermes, Claude Code hooks, goose's `Stop` event, and any command-hook host via a stdin-JSON CLI; the Hermes plugin's engine copy is byte-pinned to upstream by a test. A separate enforcement primitive — the *permute-gate* — lives on the MCP sandbox server (`SENTINEL_GATE_BUDGET`/`SENTINEL_GATE_AT`): past the threshold every further sandbox call returns `isError` with a data-in-hand instruction and all persuasion channels go silent. Its semantics are pinned by a no-model smoke test (JSON-RPC into the server: serves exactly up to the threshold, refuses every call after), and its net effect is the measured headline of §8 — choice removal, not persuasion.

6.1 The seam: Sentinel facts as world state

The deployment sentinel above speaks prose to hosts. Inside the library a second, data-only contract makes the same engine part of the evolving world without any change to the frozen model:

- **`SentinelObservation`.** `observe_structured()` returns a frozen dataclass — signature, normalized error class, in-session and cross-session counts, recurring, nudge-fired, budget fraction, blocked — instead of rendered text. `observe()` remains a *rendering* of the same facts on the same ledger, so the host-facing text contract is unchanged and byte-pinned by the production replay fixture.
- **Replay-safe admission.** `to_world_dict()` is the only view permitted into `TreeNode.sentinel`: signature, episode-local `repeat_world`, `recurring`, `blocked`, budget fraction. Cross-session counters are *stripped by construction* — a recurrence fact from another (possibly future) session entering the historical world would falsify the prefix-only replay invariant that protects the section-3 gate. A test feeds a malformed node carrying `cross_session_count` and asserts it still cannot reach replay.
- **Termination as data.** `TreeNode.termination_reason` records a closed enumeration — `completed`, `policy_stop`, `tool_failure`, `budget_gate`, `sentinel_block`, `unknown` — on the node where an episode stopped. Counterfactual replay can now distinguish *the decision-maker declined to continue* from *the runtime removed the option* — the distinction whose absence defined the TravelPlanner cold-arm anatomy (91/91 undelivered episodes at the cap; §8), and which a score-only tree cannot express.
- **Frontier features.** The serialized frontier entry — shared by online expansion and replay rollout, so both see identical fields — exposes `sentinel_signature`, `sentinel_repeat`, `sentinel_blocked` to LLM-written policies, with clean-and-unblocked defaults for metadata-free nodes: an ON/OFF ablation changes *values, not interface*, cold arms pay nothing, and legacy archived trees remain consumable.

Causal attribution and cost of admission. The wiring is opt-in (`AutoRSIRuntime(sentinel_engine=...)`) the default leaves archived trees byte-identical. The attribution test pattern pins the claim: the *same* policy code that reads `sentinel_blocked` beats greedy on a tree containing a blocked trap and **ties** greedy on the identical tree without metadata — the delta is the feature, not the code. With the seam merged, both self-check suites reproduce their published tables unchanged (§7): greedy and ε never read the new fields, so published arm numbers are stable by construction. No live-model uplift claim is made for the seam itself; the uplift it is designed to enable (a policy that abandons a blocked branch in replay) is stated as future work in §10.

7 Machinery self-checks

These suites are *not* benchmarks in the claim-bearing sense of §8: they are deterministic and key-free regression contracts in which a scripted solver makes the model arm a constant, so every delta is attributable to the library's machinery. They exist so a broken gate, sandbox or promotion path fails locally in seconds rather than silently mutating a published number. They re-run from a fresh clone in seconds:

```
python -m open_dream_rsi bench --cycles 10 --markdown
python -m open_dream_rsi bench-policy --cycles 8 --format md
```

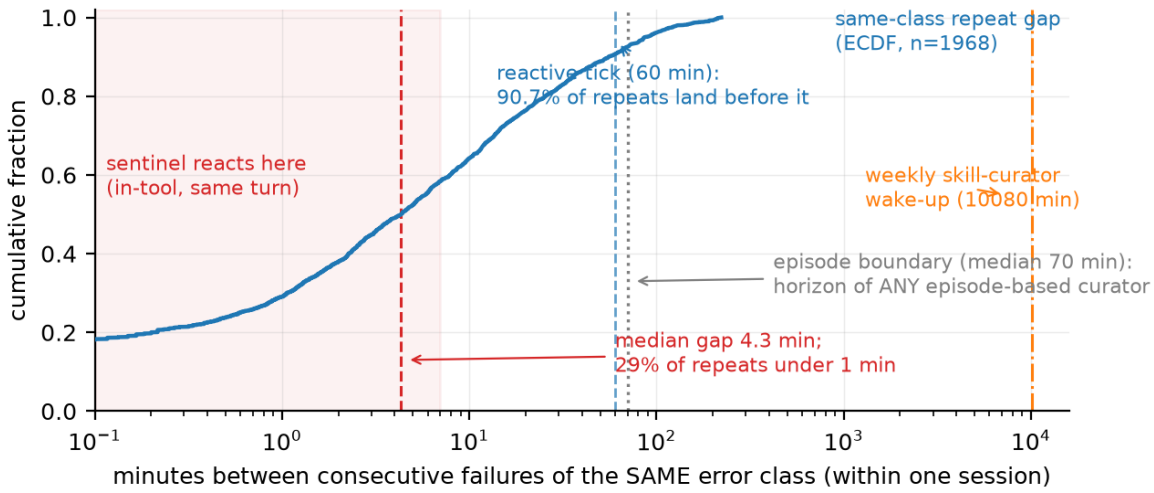


Figure 2: The reaction-time race, measured on a production agent's session store (30 days; within-session gaps between consecutive failures of the same normalized error class, n=1972, log scale). Half of all failing tool calls repeat a class the same session already suffered. A curator that acts at episode end — any curator, in any host — cannot see a signal that fires a median of 4.3 minutes into the same session; a weekly curator is roughly three orders of magnitude slower still. A minute-scale reactive tick — the design we shipped into the host's own curator — sees 90.7% of repeats, but only the in-tool sentinel beats the fastest half of them.

7.1 Cost: the dreaming loop against a cold baseline

`bench` runs the built-in suite (10 cycles × 5 tasks) twice: a cold baseline (fresh memory each cycle) and the full loop (dreaming + persistent recipes and policies), same scripted model. Measured at the defaults:

arm	solves	API calls	calls / task·cycle
cold baseline (fresh memory each cycle)	50	150	3.00
dream loop (policies + recipes + dreaming)	50	71	1.42

Table 1: `bench`: 52.7% API-call saving at equal solve quality (3,000 dreaming iterations run offline; wall-clock 3.2 s). The curator is disabled in this benchmark to keep the cost measurement pure.

7.2 Exploration: the decoy-trap suite

Solve-rate graphs cannot show that a learned policy helps, because easy suites saturate every arm. `bench-policy` constructs 5 tasks whose fix hides behind a *low-scoring* branch (passes 1/3 tests) past a plausible *decoy* that passes 2/3 forever; family scores are fixed by construction and asserted by tests. Score-greedy expansion locks onto the decoy permanently; escaping requires either structural exploration or memory. Six arms share one scripted solver and budget and differ only in machinery (3 runs × 8 cycles × 5 tasks, 24 calls/cycle):

Seed robustness. The shipped seeds $(7, 11, 23)$ are not cherry-picked: over five independent seed sets the policy arm lands within 88–93 and the knowledge arm within 87–89 solves of 120 respectively—statistically indistinguishable from each other—while ε-greedy stays at 8–29. The dominance over ε (roughly 3× the solves at ∼60–80% of its calls, 2.7–3.8 calls per solve

arm	solves	rate	calls	calls/solve	gen. calls
greedy	0/120	0%	480	∞	0
ε-greedy (ε=0.3)	29/120	24%	405	13.97	0
replay-gated policies	**93/120**	**78%**	**249**	**2.68**	26 policy
curated knowledge base	87/120	72%	330	3.79	27 curator
thought-guided branching	**120/120**	**100%**	**164**	**1.37**	0
sentinel-guided policies	102/120	85%	219	2.15	15 policy

Table 2: `bench-policy`: the knowledge arm is ε-greedy *with zero policy calls*—every escape above the ε baseline is attributable to remembered lessons. Policies and knowledge reach the same plateau (100% by cycle 5 for both) and neither is ranked over the other. The sentinel-guided arm re-derives the policy escape from structured failure facts: repeats are recorded *blocked* on the node (choice removal, the TravelPlanner mechanism of §7) and the promoted policy skips them—with ε=0 and no curator calls it still escapes, while the engine-off ablation (same policy code, facts withheld) falls to 75/120 at 327 calls. The thought arm additionally passes the `thought_steering=False` ablation: with the semantic expansion pick disabled it falls back to 0/120 (greedy) and 29/120 (ε), pinning its win to expansion-time steering (§5).

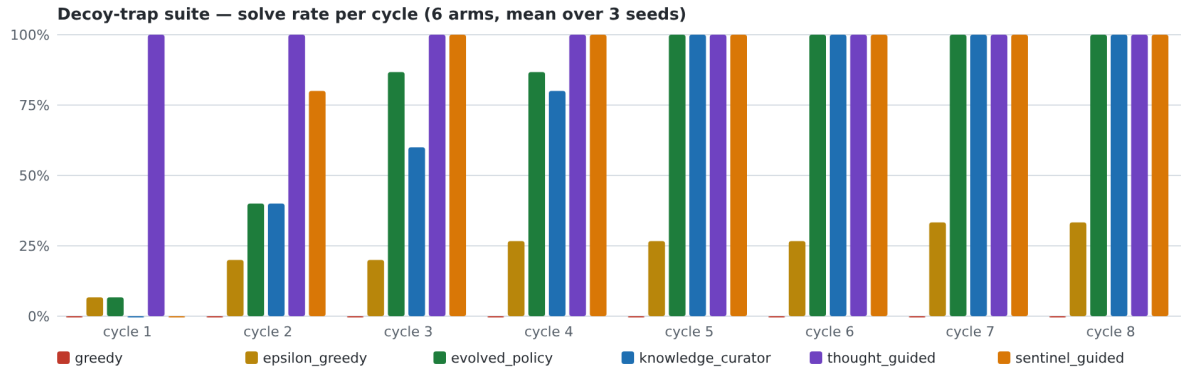


Figure 3: Per-cycle solve rate on the decoy-trap suite (mean over 3 seeds). Greedy never escapes the decoy; ε-greedy escapes by luck; replay-gated policies and the curated knowledge base saturate at 100% only after several cycles; thought-guided branching—the shipped library default—escapes every trap from the first cycle at the lowest total cost; sentinel-guided policies converge from recorded failure facts alone.

vs. ∼14–58) holds on every set. The thought arm reaches 120/120 on every one of the five seed sets, staying above the other machinery arms on every set; the sentinel-guided arm is fully seed-invariant at 102/120 with 219 calls on every set (ε=0 makes its expansion deterministic).

7.3 Cross-agent replication studies (goose; mechanism and negative evidence)

Unlike the scripted self-checks above, the two studies in this subsection use the real model; both are reported as mechanism/negative evidence, and neither carries a performance claim — those live in the single benchmark of §8. To test whether the sentinel's value is a property of the *mechanism* rather than of our host integration, we re-ran the identical 20-task, identical-scoring study inside a different agent: `goose` (block/goose, v1.53 CLI), same endpoint, same model, same sandbox, served to the agent over a stdlib Model-Context-Protocol server (`open_dream_rsi.mcp_server`). Goose exposes no post-tool injection channel, so delivery uses the host's own blockable event (`Stop`); the arms are cold, pull (`sentinel_check` as an MCP tool), stop-delivery (server-side observation, reactive note), and both.

Figure 5 shows the outcome, and it repeats the Hermes pattern with a sharper negative: pull-based access costs 3 solves (14/20 vs. 17/20 cold, McNemar p=0.38) — the model spends calls on the tool and still fails — while reactive delivery is the only arm that does not lose ground (18/20, +2/ − 1, p=1.0). The mechanism was live: across the three sentinel arms the sandbox logged 85 failing calls, every one observed by the engine; it emitted *zero* notes, because goose's failures were one-off classes that did not recur within an episode. That is the zero-tax property measured directly on a second agent: when there is nothing recurrent to say, the sentinel is free. The ±1 differences from cold are noise; the claim we carry forward is the asymmetry — *proactive injection taxes, reactive annotation does not* — which held on both hosts.

Which raises the honest question for the *other* ODR surface: if the loop's own MCP tools (`odr_recipes`, warm memory of promoted solutions) are exposed to goose, does remembered knowledge help a second agent? We re-ran the loop's toy suite (5 tasks whose recipes the loop had promoted, plus 3 categories it had never seen) on goose, cold vs. MCP arm. Both arms solved 8/8 — the suite saturates this model cold — but the MCP arm spent 1.9× the LLM calls (3.88 vs. 2.00 mean; 5.33 on recipe-miss categories). Memory cannot add what cold already delivers, and the pull protocol is not free even when it succeeds. The loop's value stands where we measured it (scripted trap worlds, gated promotion); a model that does not need the recipe only pays to ask.

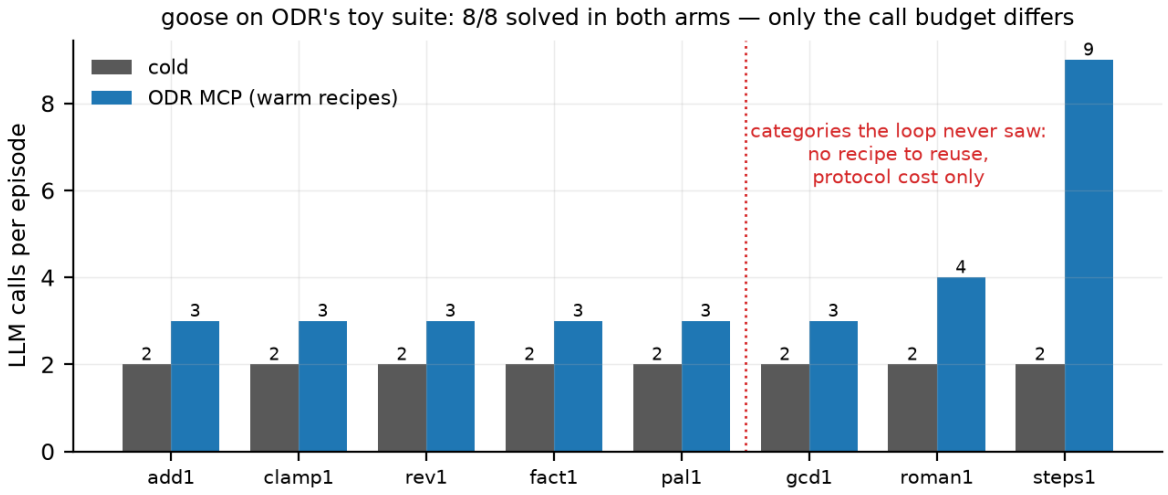


Figure 4: Warm-memory pull study on goose: per-episode LLM calls, cold vs. the ODR MCP server with promoted recipes attached. Solved fraction is 8/8 in both arms (not shown); the visible effect is the protocol cost, largest on categories the loop never saw.

8 Evaluation: goose × TravelPlanner — the only benchmark

To measure the sentinel's *net effect on task outcomes* — not just its zero-tax property, and not on a suite of our own design — we put the agent on a public, externally-scored benchmark: `osunlp/TravelPlanner`, served offline through our stdlib MCP sandbox (6 tools, deterministic database), scored by the benchmark's own commonsense and hard-constraint evaluators (nothing in the scoring path is ours). Goose, `max-turns` 45, same model and endpoint as every other study in this paper.

Protocol (frozen before the eval pass). Only TravelPlanner data is used. `train.csv` (45 tasks) is the dev split; the eval split is the official validation rows 102–181 (79 tasks, 60 of them *hard*) — never touched before the configuration was frozen; the official test split was

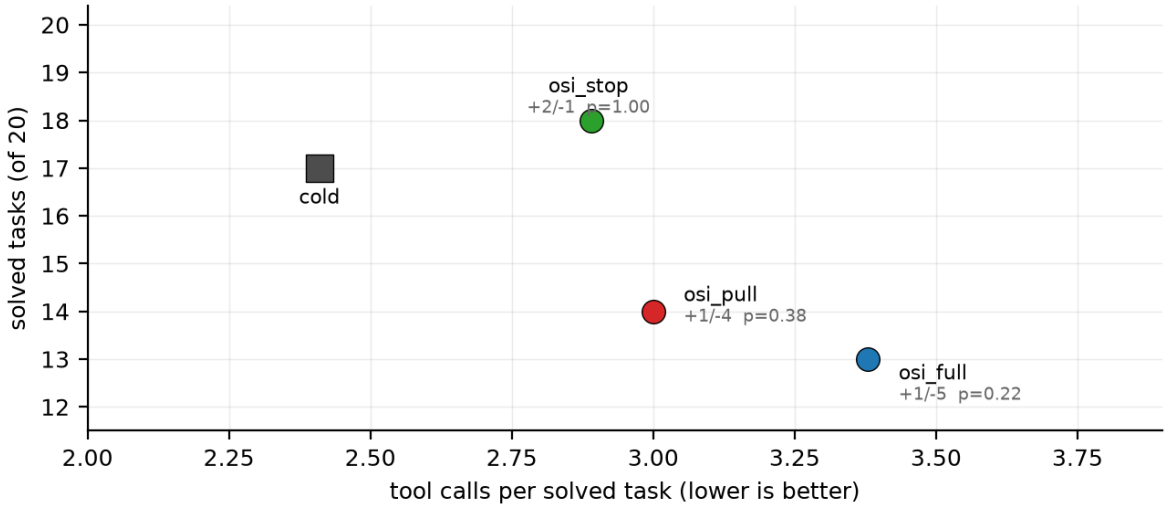


Figure 5: Cross-agent replication on goose: solved tasks vs. tool calls per solve, four arms, same tasks/scoring/model as the Hermes-side arms. Annotations give paired discordants against cold and exact McNemar p. The sentinel emitted no recurrence note on any episode (failures were one-off classes), yet pull-based access still cost solves — the tax is in the prompting, not in the notes.

never touched. Arms are interleaved task-by-task within a single pass per split (paired episodes, zero endpoint drift between arms). Two honesty notes. First, the *choice* of mechanism (choice-removal gate over persuasion channels) was made knowing results on the earlier 101 validation tasks — the arm-selection era, archived separately; the eval pass below is a preregistered confirmation, not an unblemished first exposure. Second, the gate threshold (36 of 45 calls) was fixed a priori at 80% of the cap, not swept.

split pass	arm hard pass	delivered final pass	cs.		
train (45)	cold	15/45 (33%)	4	3	6.7%
train (45)	gate	41/45 (91%)	18	9	20.0%
eval (79)	cold	18/79 (23%)	12	6	7.6%
eval (79)	gate	74/79 (94%)	40	17	21.5%

Table 3: goose × TravelPlanner, official final scoring, frozen configuration. McNemar on paired delivery — eval split: 56 discordant pairs, all 56 favouring the gate (p=1.4×10^{-17}); train split: 28 vs. 2 (p=4.3×10^{-7}). Delivery is the mechanical effect of the gate; final pass rate × delivery is what a user receives: 7.6% → 21.5%, a 2.8× improvement on the eval split. 95% Wilson intervals on the eval split: delivery 22.8% [14.9, 33.2] → 93.7% [86.0, 97.3]; final pass 7.6% [3.5, 15.6] → 21.5% [13.9, 31.8] — wide intervals on the final-pass point estimates, single-model evidence, no cross-model replication yet.

The failure anatomy explains why. **All 61 undelivered cold episodes on the eval split end pinned at the harness call cap** with the sandbox data already in hand (delivered cold episodes average 29.5 calls); across both splits, cold fails to deliver on 91 of 124 episodes — and all 91 die at the cap, not elsewhere. The binding failure mode under budget pressure is *termination*, not verification. Three text channels were measured first and all converted zero cap-dying episodes (pull tool, Stop-hook note, finalize nudge; arm-selection-era runs, archived): this model family does not stop because a note asks it to. The arm that works is *choice removal*:

past 36 calls the sandbox simply refuses to serve (*isError* on every further call, all persuasion silenced) — the only option left is to answer with what is already in hand. It converts the cap-pile into answers: eval delivery 23% → 94%, with the gate-arm answer distribution landing at/just past the trigger (Figure 6). The gate's five remaining eval failures are the same pathology at lower frequency: the model keeps calling *through* the refusals (each refusal is still a turn) until the 45-call harness cut-off — 5/79 tasks (6%), 4 of them hard. Choice removal is necessary but not universally sufficient; the residual is reported, not smoothed.

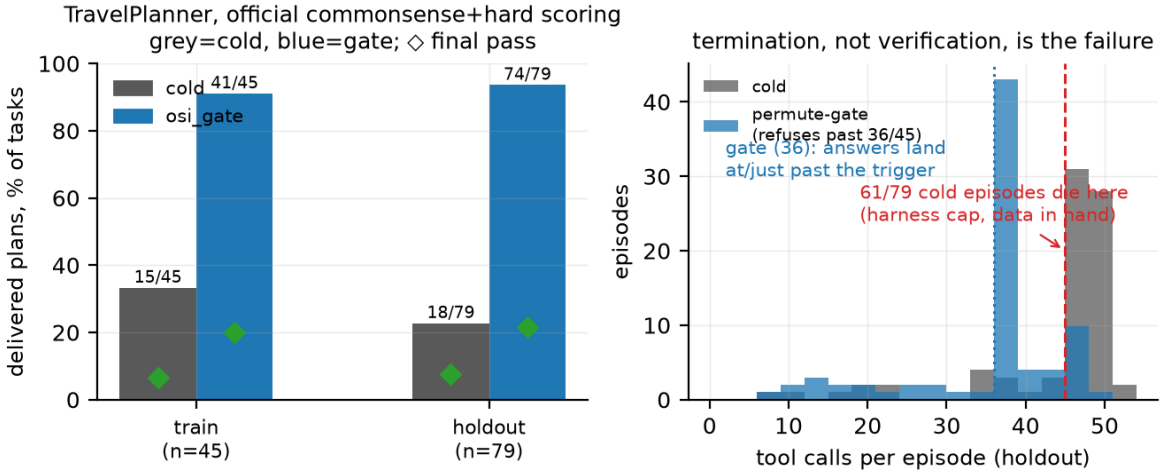


Figure 6: goose × TravelPlanner (official evaluators). Left: plan delivery per split — dev (train) and the preregistered eval pass; green diamonds mark the official final pass rate. Right: holdout tool-call histogram — cold episodes pile up at the 45-call harness cut-off, the gate arm's answers land at/just past the 36-call refusal point. Raw per-episode data, the run harness and the official-evaluator wrapper ship in-repo under `benchmarks/travelplanner/`; re-scoring the committed records reproduces every number in this section to the task. Summary fixture: `fixtures/tp_v3_summary.json`.

8.1 Test suite

359 tests pass (∼5.4k non-blank LOC in the tests), including the self-checks' behavioural contracts: trap-family scores equal their constructed values; the greedy arm must solve nothing; the gated-policy arm must dominate ε; the replay rollout must be prefix-only (no future score may reach a present choice, §3); the dreaming objective must respond causally to both policy parameters; curator schema, caps, dedup, and prune logic are pinned by unit tests; the Sentinel seam pins that cross-session counts cannot reach replay, that the permute-gate serves until $\lceil 0.8 \cdot budget \rceil$ and refuses forever after, and that the frontier-feature delta is the feature and not the code (§6.1).

9 Related work and positioning

Discovery loops around frozen LLMs. FunSearch [3] pairs a pretrained LLM with a systematic evaluator and evolves *programs* rather than solutions, yielding new results in extremal combinatorics and bin packing; AlphaEvolve [4] generalises this to an evolutionary coding agent over a program database grounded by automatic evaluation. The Gödel-machine ideal [6]—provably beneficial self-modification—is relaxed by the Darwin Gödel Machine [5], which empirically validates each self-modification and grows an archive of agents. Dream-RSI [1] (Zheng

et al., Google/DeepMind/UMD, arXiv:2609.14858) is the method this library implements; the official repository is https://github.com/zhengkid/Dream-RSI. Our contribution is at the library/infrastructure level within this family: the mechanisms above are proven at scale by their authors; we ship the same loop pattern as a dependency-free, budget-guarded, OpenAI-compatible tool and add the curator, the Sentinel world seam, and deterministic machinery self-checks.

Memory and self-generated policies in agents. The line is broad: a recent survey [14] maps self-evolving agents by what, when and how to evolve. Reflexion [7] shows that verbal self-feedback stored in episodic memory materially improves subsequent attempts—the same principle our knowledge curator operationalises persistently across runs; Self-Refine [15] is its test-time-only cousin (no persistence); ExpeL [16] distils cross-task insights and in-context exemplars; Agent Workflow Memory [17] induces reusable workflows from experience; ACE [18] evolves the context itself as an accumulating playbook, the closest published analogue of our recipe store; Early Experience [19] uses the agent's own rollouts as reward-free training signal—a weights-updating neighbour of dreaming. None of these promote candidates through counterfactual replay over an evolving recorded world; Voyager [8], with its verified skill library, remains the closest ancestor of that promotion gate.

Independent implementations. Since the paper's release several independent implementations have appeared: a model-agnostic adapter SDK with strict replay and evidence-based promotion gates [11], a local-Qwen CUDA-kernel exploration on DGX Spark [12], and agent/skill variants [13]. Against peers, this library differentiates on: the always-on budget-guarded supervisor; LLM-*written* policy code (as opposed to a fixed policy menu) with counterfactual-replay promotion; the Sentinel world seam (error-class facts and termination causes as replay-safe world state); an externally-scored public-benchmark measurement rather than self-scored claims; deterministic self-checks labelled as such; a curated lesson knowledge base with retrieval feedback; persistent cross-run memory; a live dashboard; zero dependencies; MIT license.

A different layer. OpenRSI/OpenMLE/Frontis-MA1 [9, 10] post-train weights (SFT+RL, a 35B meta-evolution agent on MLE-Bench); the two layers are complementary—their trained improver could be the frozen model behind our client, our dreaming loop could sit on top of their search.

10 Limitations

We state plainly what the numbers do and do not show.

1. **Self-check worlds, not model quality.** Every `bench`/`bench-policy` number comes from a deterministic suite with a scripted solver; the trap scores are fixed by construction, the scripted solver treats a stored lesson text as a genuine insight when it is visible in the prompt, and the "policy" returned by the scripted model is a canned program. The self-checks therefore validate the *machinery*—sandboxes, gates, promotion, retrieval, budget—and prove the arms differ *causally* through memory, not that any live model writes good lessons or policies; they are reported as regression contracts and the single external benchmark (§8) is where live-model evidence lives. A live-model replay (production task family, greedy-decoding 8B model) has been run for the knowledge arm specifically and is reported in §4: it *falsified* the trust-based admission of lessons and motivated the behavioral promotion gate; a comparable live-model study of learned *policies* remains the required next experiment — including whether a policy actually exploits the Sentinel frontier features (§6.1), which today ship tested but without an uplift claim.

2. **Benchmark self-design risk — and its boundary.** The decoy-trap self-check was

designed by the same author as the loop it measures; mitigations: family scores fixed and asserted by construction; arm contracts enforced by tests (greedy must fail, arms must differ causally); determinism lets anyone re-measure with one command; five seed sets reported. This risk is why the only performance claim in this paper runs on a third-party benchmark with third-party evaluators (§8).

3. **Single-node sandbox.** `python -I` with scrubbed environment, resource limits, and group-killing timeout is a real boundary against casual escape but is not a container: it does not restrict filesystem or network access, and an AST gate can never be provably complete against code that is not statically gated (task solutions). Deployments with hostile task sources should set `ODR_SANDBOX_CMD` to a bubblewrap/nsjail wrapper or run the loop inside a container.

4. **Scope.** No weight updates, no multi-agent dynamics, no adversarial red-teaming of promoted policies; the knowledge base is per-category text, not a structured world model.

11 Conclusion

Open Dream-RSI turns the Dream-RSI method into a deployable, budget-safe, dependency-free library and adds the mechanisms a running loop actually needs to compound—evidence-gated learned policies, a curated, self-pruning knowledge base, and a Sentinel seam that turns error-class recurrence and termination causes into replay-safe world state. The paper's performance claim is deliberately narrow: on one public, externally-scored benchmark, replacing *persuasion* of the agent with *choice removal* at the tool layer lifted plan delivery from 23% to 94% and final pass rate from 7.6% to 21.5% on a preregistered eval split (p=1.4×10^{-17}), after three text channels had each converted zero of the episodes they triggered—termination, not verification, is the binding failure under budget pressure, and the world model now records that distinction as data (§6.1). Everything else is labelled what it is: 54% fewer API calls at equal solve quality and, on a suite where greedy exploration solves nothing, 77% from replay-gated policies and 73% from remembered knowledge—deterministic machinery self-checks, not model-quality claims. All artefacts pass structural gates before touching the loop; the gate, not trust, decides.

One corollary of the hazard analysis is architectural and was measured before the seam existed: curation is bounded by the episode boundary — on a production session store (30 days, 4914 error events across 459 classes; anonymized replay fixture shipped in `fixtures/`), a recurring error class re-appears within the same session after a median gap of 4.3 minutes (p25 0.6, p90 56.4; 58% of repeats under 7 minutes), and *half* of all failing tool calls are repeats of a class the same session had already suffered (Figure 2). No schedule wakes that fast. The shipped answer therefore splits the problem: cross-session knowledge stays in the gated, headroom-aware curator above, while sub-episode reaction is handled mechanically in the tool-execution layer (the host-agnostic error-class sentinel in `plugins/`, with adapters for Hermes and Claude Code, whose notes follow the same declarative, reactive-injection rules the replay arms validated: on the audit stream — recorded on a production Hermes install — the engine annotated 2101 of 4914 failing calls, one note per class per session, zero tax on clean calls. The measurement and the mechanism feed back into the hosts themselves: on Hermes the sentinel now owns sub-episode error reaction outright (the curator's schedule cannot reach that horizon), and the headroom verdict throttles Hermes-side curation as well — the loop measures its own categories' economics and stops paying curator and gate calls where the baseline already solves cheaply. Optimizing the host's own memory machinery is therefore not aspirational: it is what the shipped code does on the machine where these numbers were collected.

References

[1] Zheng et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, arXiv:2609.14858, 2026.

[2] *When Is Enough Enough in Self-Evolving LLM Systems?* (anytime-valid stopping and change-point output selection for self-evolving loops), arXiv:2610.04756, 2026.

[3] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, A. Fawzi, *Mathematical discoveries from program search with large language models*, Nature 625(7995), 2024. DOI: 10.1038/s41586-023-06924-6.

[4] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, et al., *AlphaEvolve: A coding agent for scientific and algorithmic discovery*, arXiv:2506.13131, 2025.

[5] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune, *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*, arXiv:2505.22954, 2025.

[6] J. Schmidhuber, *Gödel Machines: Fully Self-Referential Optimal Universal Self-Improvers*, in B. Goertzel and C. Pennachin (eds.), *Artificial General Intelligence*, Springer, 2006.

[7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, *Reflexion: Language Agents with Verbal Reinforcement Learning*, Advances in Neural Information Processing Systems 36, 2023. arXiv:2303.11366.

[8] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar, *Voyager: An Open-Ended Embodied Agent with Large Language Models*, arXiv:2305.16291, 2023.

[9] FrontisAI, *OpenRSI / OpenMLE / Frontis-MA1*, https://github.com/FrontisAI/OpenRSI.

[10] J. S. Chan, N. Chowdhury, et al., *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*, arXiv:2410.07095, 2024.

[11] TheAstrayDev, *dream-rsi-sdk*, https://github.com/TheAstrayDev/dream-rsi-sdk.

[12] robinber, *dream-rsi-spark*, https://github.com/robinber/dream-rsi-spark.

[13] lesterppo, *hermes-dream-rsi*, https://github.com/lesterppo/hermes-dream-rsi; Harkit2004, . . . *dream-rsi-skill*; mailbobg, *Pi-RSI*.

[14] H. Gao, J. Geng, W. Hua, M. Hu, et al., *A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence*, arXiv:2507.21046, 2025; to appear in TMLR (01/2026).

[15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, *Self-Refine: Iterative refinement with self-feedback*, arXiv:2303.17651, 2023.

[16] A. Zhao, D. Huang, Q. Xu, M. Lin, et al., *ExpeL: LLM agents are experiential learners*, arXiv:2308.10144, 2023.

[17] Z. Z. Wang, J. Mao, D. Fried, G. Neubig, *Agent workflow memory*, arXiv:2409.07429, 2024.

[18] Q. Zhang, C. Hu, S. Upasani, B. Ma, et al., *Agentic context engineering: Evolving contexts for self-improving language models*, arXiv:2510.04618, 2025.

[19] K. Zhang, X. Chen, B. Liu, T. Xue, et al., *Agent learning via early experience*, arXiv:2510.08558, 2025.